

Subscription App Retention & Pricing Case Study

Oct 2, 2026 · by @Menna

Executive Summary

A June 2026 price increase on the monthly plan (\$12.99 to \$15.99) drove a real, measurable churn spike: active monthly-plan users in the pre-hike cohort fell 23% in the month immediately following the hike and stayed down, while annual-plan users, unaffected by the price change, declined only 4-6% over the same period. The effect is specific to monthly-plan users, corroborated by a spike in billing-complaint support tickets (676-780 tickets a month in June-July, versus single digits for annual users), and a flat ~5% failed-payment rate rules out a technical billing bug as an alternative explanation.

Revenue tells a more nuanced story. Company-wide monthly-plan revenue climbed every month through August, but that growth came entirely from new customer acquisition. Revenue from the existing pre-June cohort peaked the month of the hike, then fell back to pre-hike levels by August — the price increase was a net wash for customers who were already subscribed.

Retention also varies meaningfully by acquisition channel: referral-acquired users retain 76% of their original cohort by month 5, versus just 53% for influencer-acquired users.

Business Context & Questions

This is a simulated dataset with no real company, customers, or transactions involved, spanning 2026-03-01 to 2026-08-31: 15,000 users signed up through four acquisition channels (organic, referral, paid_search, influencer) onto either a monthly or annual plan, across ios, android, and web. The dataset covers users (signup, channel, plan, device), usage events (app opens, content views, feature use), charges (billing period, amount, status), and support tickets (category, resolution time).

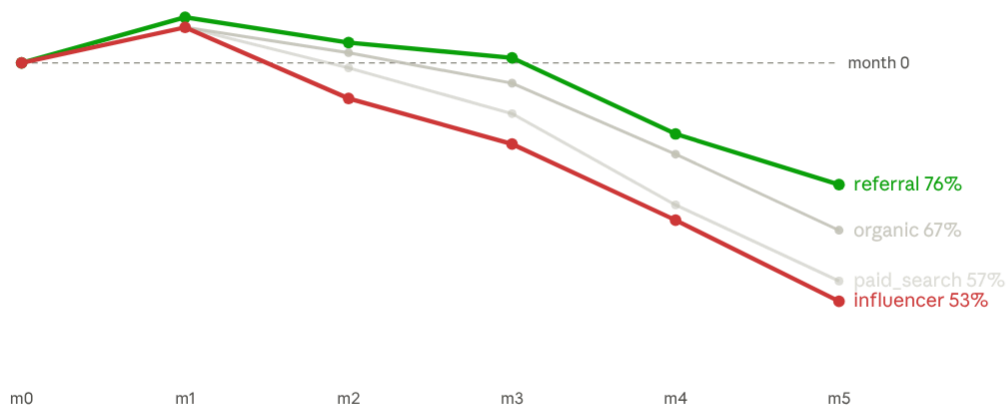
The analysis is built on four questions:

1. Which acquisition channel retains users best?
2. Is there a specific point where users commonly churn, and does it line up with a business event like a price change?
3. How is monthly recurring revenue trending over time?
4. Do support tickets corroborate whatever pattern is found in question 2?

Finding 1: Acquisition Channel Retention

Referral retains best, influencer worst

% of March cohort still active, by months since signup



SQLite query against the simulated dataset · March 2026 signup cohort, 6 months

Referral-acquired users retain best over time, while influencer-acquired users churn fastest, tracking the March 2026 signup cohort across its first six months. By month 5, referral retains 76% of its original cohort, organic 67%, paid_search 57%, and influencer just 53% — a 23-point gap between the best and worst channel. The likely explanation: referral users arrive already vouched for by someone they trust and with accurate expectations, while influencer-driven signups tend to be more impulsive and a weaker product fit.

The brief rise above 100% in the first month or two is a cohort artifact rather than growth: some March signups don't log their first usage event until April or May, so the active count within the cohort keeps climbing before the expected decline sets in.

Finding 2: Price Hike-Driven Churn

Monthly-plan users cratered after the price hike

% change in active users vs. the prior month, pre-June signup cohort



SQLite query against the simulated dataset · pre-June 2026 signup cohort, April-August

Monthly-plan users saw their price rise from \$12.99 to \$15.99 starting June 2026; annual-plan users, who pay one lump sum up front, were unaffected. Watching the same pre-June signup cohort before and after, monthly-plan active users fell 23.1% the month after the hike and stayed down (-23.6% the month after that), while annual-plan users declined only 3.9% and 5.7% over the same two months — a mild, normal drift rather than a cliff. Two alternative explanations were ruled out: a general product issue would have hit annual users just as hard, and a technical payment bug would have shown up as a spike in failed charges, but the failed-charge rate for monthly-plan users held flat around 5% through the same months. The price increase itself is the best-supported explanation for the churn.

Finding 3: Revenue Impact of the Price Hike

New signups mask a revenue loss on the existing cohort

Monthly-plan revenue collected per month (USD)



SQLite query against the simulated dataset · monthly-plan charges, March-August 2026

Company-wide monthly-plan revenue (all users, including new signups after June) climbed every month through August, which looks like an unambiguous win for the price increase. But isolating just the pre-June signup cohort — the people who actually experienced both the old and new price — tells a different story: their revenue peaked the month of the hike (\$63,480 in June), then fell to \$48,242 in July and \$37,241 in August, landing almost exactly where it was back in April (\$37,216), before the price ever changed. The price increase produced a short-lived revenue bump that churn fully erased within two months. Total company revenue kept growing only because new customers kept signing up at the higher price — the existing-customer loss was masked, not fixed, by continued acquisition.

Finding 4: Support Ticket Corroboration

Nearly the entire billing-complaint spike in June and July comes from monthly-plan users specifically, confirming the churn in Finding 2 is tied to the price change rather than a broader issue:

Month	Monthly-plan billing complaints	Annual-plan billing complaints
2026-06	676	2
2026-07	780	5
2026-08	122	9

676 of 678 total billing complaints in June, and 780 of 785 in July, came from monthly-plan users. Annual-plan users, whose price never changed, generated only single-digit complaint counts across the same months. This rules out a broader, plan-agnostic product issue and confirms the complaint spike is concentrated on exactly the group the price hike affected.

By August the gap narrows as complaint volume fades for both groups (122 monthly vs. 9 annual) — consistent with the affected cohort settling into the new price or having already churned out.

Recommendations

- **Pricing:** a flat, unannounced price increase on the monthly plan produced a short-lived revenue bump that churn erased within two months. Future price changes should be modeled against expected churn before launch, and consider a smaller or phased increase, advance communication, or grandfathering existing subscribers.
- **Retention:** monitor active-user counts for the existing customer base separately from total company metrics after any pricing change — new-customer growth can mask a real loss among existing customers for months.
- **Channels:** invest more in referral acquisition (76% retention at month 5) and investigate why influencer-acquired users churn fastest (53%) — likely a targeting or expectation-setting problem rather than a product problem.
- **Support monitoring:** billing-complaint volume moved in lockstep with the churn it was warning about. A real-time alert on category-level ticket spikes (not just overall volume) would have surfaced this risk within days of the price change, rather than requiring a retrospective analysis.

Tools & Methodology

This analysis was conducted entirely in SQL against a simulated SQLite database — no real company, customers, or transactions are represented in this dataset.

Techniques used across the analysis:

- **CTEs** — multi-step cohort construction (activity-by-month, months-since-signup, retention counts) built as layered, named steps
- **Window functions** — `LAG()` for month-over-month comparisons within each channel/plan-type partition
- **Conditional aggregation** — `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` to compute failed-charge rate and other rates needing two counts from the same row set

- **Joins** — a deliberate mix of `INNER JOIN` (price-hike churn, which only needs users with recorded activity) and `LEFT JOIN` (channel retention, revenue, and support-ticket queries, which keep every row from the primary table regardless of a match), chosen per query based on which table's rows needed to be guaranteed in the result
- **Date arithmetic** — `SUBSTR()` for month truncation; `CAST(...AS INTEGER)` month-difference math for cohort-relative time
- **Aggregate functions** — `COUNT(DISTINCT ...)`, `SUM()`, with explicit float-division handling (`*1.0`) to avoid integer-division truncation

Every finding in this report was checked against real query output before being written up, including two cases where an initial query would have produced a misleading result (counting events instead of distinct users; mixing all ticket categories instead of isolating billing complaints) and was corrected before being used.